

C106 - D

15

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 - ANALYTICAL PROBLEM SOLVING

Course Code: CSA1407 Course Title: COMPILER DESIGN
 CO Assessed: CO1 - Imitation (L1) Assessment: Assessment Tool 1 - ANALYTICAL PROBLEM SOLVING
 Weightage: 40% of CIA
 CO1 Statement: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)
 Student Name: K. Sabitha Reg No.: 192524303
 Section / Batch: Date: 16/7/2026

Code of Conduct

I, K. Sabitha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student K. Sabitha

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes

Q No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 / 4	3 / 4	4 / 4	4 / 4	3 / 4	91.25
2	4 / 4	4 / 4	3 / 4	4 / 4	4 / 4	93.75
3	3 / 4	4 / 4	3 / 4	4 / 4	4 / 4	88.75
4	4 / 4	4 / 4	3 / 4	3 / 4	3 / 4	85
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	89.75
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N. D. J.

Compiler - Compiler Design
CO1 - Measurement test

a) Lexical Analysis

Tokens Generated

$$P = (a * b) + (b * c) * 2$$

Identifier = P, a, b, c

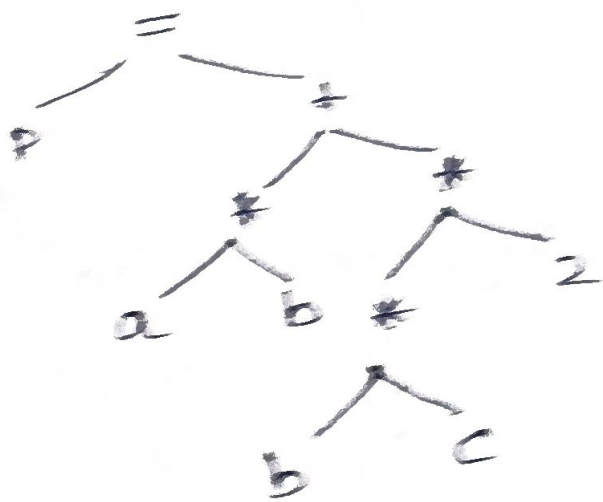
Operator = *, =, +

Parenthesis = ()

Constant = 2

Token 3 = 16

b) Syntax Analysis



c) Intermediate Code Generation

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_3 = t_2 * 2$$

$$t_4 = t_1 + t_3$$

$$P = t_4$$

d) code optimization.

$t_1 = a * b$

$t_2 = b * c$

$t_3 = t_2 << 1$

$P = t_1 + t_3$

e) code Generation.

MOV R1, a

MUL R1, b

MOV R2, b

MUL R2, c

SHL R2, 1

ADD R1, R2

MOV P, R1

2) (a) $(\epsilon + aa^*)$

Accepted string

ϵ

a

aa

aaa

aaaa

Rejected string

b

ab

ba

bb

aab

b) $(a + b^*)abb$

Accepted string

aabb

abb

babb

bbabb

bbbabb

Rejected strings

ab

aab

ba

abab

bba

c) $(a * b)^* aba(a+b)^*$
Accepted strings

aba
abaa
abab
baba
aababa

Rejected string

aa
abb
baa
bab
aaaa

d) $(a+ab)^*$

Accepted strings

a
ab

Rejected strings

aa
abb
b
ba
aba

e) $(aba)^+ (a * b)^*$

Accepted strings

aba
abaaba
abab
abaqab
abaaab

Rejected strings

a
ab
baa
bba
qaab.

3 (i) main ()

{

int a = 10;

char b = "abc";

int c = 30;

char d = "xyz";

int m = 40.5;

};

Tokens = 30

Keyword : int, char, int, char, int $\rightarrow$ 5

Identifiers : main, a, b, c, d, m $\rightarrow$ 6

Operators : = $\rightarrow$ 5

Constant : 10, 30, 40.5, "abc", "xyz" $\rightarrow$ 5

Separators : () { } ; : , $\rightarrow$ 9

(ii) Identify the lexical errors.

```
void main()
{
    int x=10, y=20;
    char *a;
    a = &x;
    x = 1xab;
}
```

Correct Program

```
void main()
{
    int x=10, y=20;
    char *a;
    x = 100;
}
```

Lexical errors

1. 1xab

- * Invalid numeric literal

- * Identifiers cannot start with a digit

2. a = &x

- * Lexically valid

- * (semantic issue : assigning an integer address to a character pointer.)

4) i) Even number of 0's

$$1^* (01^* 01^*)^n$$

Accepted strings

ϵ
11
00
0011
1100

Rejected strings

0
10
100
000
1010

(ii) At least one 1

$$(0+1)^* 1 (0+1)^*$$

Accepted Strings

1
10
01
101
111

Rejected strings

ϵ
0
00
000
0000

(iii) Odd number of both 0's and 1's

$$(0(0^* 10^*)^* 1) + (1(01^* 01^*)^* 0)$$

Accepted strings

01
10
0001
0111
11000

Rejected strings

ϵ
0
1
00
11

iv) start with 1 and ends with 0.

$1(0+1)^*0$

Accepted strings

10
100
110
1010
1110

Rejected strings

0
1
101
0110
111

v) No consecutive 1's

Accepted strings

ϵ
0
1
1010
10010

Rejected strings

11
111
110
1011
1110

6) Input

int var123 = 12345;

float var456 = 123.45;

if (var123 == var456) {return};

a) Number of Tokens.

Total Tokens = 20.

b) Categorize the Tokens.

keywords: int, float, if, return

identifiers: var123, var456

operators: =, ==

Integer Literal : 12345

Floating Literal : 123.45

c) FSM State Transitions (Approximate)

* Keywords $\rightarrow$ 20

* Identifiers $\rightarrow$ 28

* Integer Literal $\rightarrow$ 6

* Floating literal $\rightarrow$ 8

* Operators $\rightarrow$ 4

* Separators $\rightarrow$ 7

Total Estimated FSM Transitions = 73.